Readable code: write for the next reader

Names, constants, small functions, comments and spacing, with every rewrite run to check its output.

THE QUESTION

Two programs print the same thing. Why is one easier to read, check and change?

THE INTUITION

Python needs few essentials, such as conditions and loops. Much of the rest, from `for` loops to good names, is there for the people who read code.

KEY TAKEAWAY

Name things for what they hold, give each job a function, let comments say why, then rerun: same output.

1 One program, rewritten: the same output, easier to read

before.py: it works, but you have to decode it

```
1 l=[72,35,90,41,28,66] 1
2 a=[]
3 i=0
4 while i<len(l): 4
5   if (l[i]>=40)==True: 2 5
6     a.append(l[i])  # append l[i] to a 7
7   i=i+1 6
8 print(len(a),sum(a)/len(a))
```

output `4 67.25`

after.py: the same job, readable

```
1 PASS_MARK = 40  # fixed by the exam rules 2 7

4 def passing(marks): 3
5     """Return the marks at or above PASS_MARK."""
6     return [m for m in marks if m >= PASS_MARK] 4

9 marks = [72, 35, 90, 41, 28, 66] 1
10 passed = passing(marks)
11 print(len(passed), sum(passed) / len(passed))
```

output `4 67.25` the same: both were run

what changed

1. names say what they hold: marks, passed
2. the bare 40 becomes the constant PASS_MARK
3. one job, one function, with a docstring
4. a comprehension replaces while, i and append
5. no == True: >= already gives True or False
6. 4 spaces per level, spaces around operators
7. the comment says why, not what

8 lines before, 7 after, blank lines
not counted: clearer is the goal, not shorter

Readable code shows what it does without making you run it in your head. The notes split Python into **essential features** (conditions, loops) and **extra features** that cut errors and help readers: any `for` loop could be a `while` loop, but the `for` is shorter. A **list comprehension** `[m for m in marks if m >= PASS_MARK]` builds a list in one line: take each `m`, keep it if the test holds.

2 Names say what a value holds

instead of	write	because it says
`l`	`marks`	what it holds (and l looks like 1)
`flag`	`is_prime`	a True/False name reads as a question
`40`	`PASS_MARK`	a constant: CAPITALS, set once at the top

A magic number, then the same rule with a name

```
seconds = days * 86400                  what is 86400?
SECONDS_PER_DAY = 24 * 60 * 60
seconds = days * SECONDS_PER_DAY        both give 259200
```

A built-in's name reused: sum now names 75, a number

```
sum = 40 + 25 + 10
print(sum([1, 2, 3]))                   Python stops here
```

```
TypeError: 'int' object is not callable
```

A **descriptive name** says what a value means; use **snake_case**: lower case, words joined by _. A **constant** goes in CAPITALS at the top (Python does not enforce it). Name every **magic number**, a bare value you must guess at. Never reuse a **built-in name** such as `sum`: Python finds yours first (the scope sheet).

3 A small function does one job, and names it

eight lines to say one thing

```
def is_teen(age):
    if age >= 13:
        if age <= 19:
            return True
        else:
            return False
    else:
        return False
```

one line, the same answers

```
def is_teen(age):
    """True for ages 13 to 19."""
    return 13 <= age <= 19
```

checked: equal for every age 0 to 29

`13 <= age <= 19`
a chained comparison: both parts must hold

```
print([age for age in ages if is_teen(age)]) → [13, 19, 16]
assert is_teen(13) and not is_teen(20)  passes: no output
```

A function should do one thing you can say in a few words; if the description needs "and", split it. Here the rule "13 to 19" hides in nested `if`s that return True or False. A comparison already is True or False, so return it (more such rewrites on the nested conditions sheet). A short function is easy to test with `assert` (the functions sheet).

4 Comments say why, not what

what: repeats the code, so delete it

```
n = len(rows) - 1   # subtract 1 from len
for r in rows[1:]:  # loop over rows[1:]
```

why: says what the code cannot, so keep it

```
n = len(rows) - 1   # row 0 is the header
for r in rows[1:]:  # skip the header row
```

help(passing) prints the docstring:

```
Help on function passing in module __main__:
passing(marks)
    Return the marks at or above PASS_MARK.
```

A **comment** runs from # to the line's end; Python skips it. One that repeats the code goes stale when the code changes; write what the reader cannot see, such as why a row is skipped. A **docstring**, the string on a function's first line, says what it gives back.

5 Spacing follows one style guide: PEP 8

rule	before.py	PEP 8 fix	times
E225	`i<len(l)`	`i < len(l)`	7
E231	`[72,35,90]`	`[72, 35, 90]`	6
E111	`··if`	`····if`	2
E226	`sum(a)/len(a)`	`sum(a) / len(a)`	2
E712	`(x>=40)==True`	`x >= 40`	1
E741	`l`	`marks`	1

```
ruff check before.py → Found 19 errors.
ruff check after.py → All checks passed!
```

ruff 0.12.1 is a style checker; · marks a space

a tab and 8 spaces in one block

```
1 if True:
2 ⟶   x = 1          tab
3 ········ y = 2     spaces
```

```
TabError: inconsistent use of tabs and spaces in indentation
```

`x=3*2+1 and x = 3 * 2 + 1`
both give 7: spaces never change it

PEP 8 is Python's style guide: 4 spaces per level, spaces around =, comparisons and (usually) arithmetic, a space after each comma, lines of at most 79 characters, two blank lines around a top-level `def`. A **style checker** such as ruff or flake8 lists every break. Python ignores spaces inside a line, not at its start.

Sets: each value once, in no order, found fast

Making sets, why they have no order, why `in` is fast, changing a set, and the four set operations.

6 Braces make a set; `{}` makes a dict

```
{2, 3, 5, 7, 11}              → {2, 3, 5, 7, 11}
set([3, 1, 3, 2, 1])          → {1, 2, 3}            repeats dropped
set("hello")                  → {'e', 'h', 'l', 'o'}  one 'l'
set()                         → set()                the empty set
type({})                      → <class 'dict'>       {} is a dict!
{2, 3, 5} == {5, 3, 2}        → True                 order ignored
sorted(set([3, 1, 3, 2, 1])) → [1, 2, 3]            a sorted list back
```

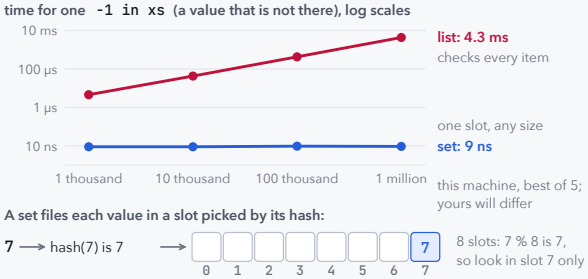
`print({"pear", "fig", "kiwi", "plum"})` in three fresh runs:

```
run 1  {'pear', 'fig', 'kiwi', 'plum'}
run 2  {'fig', 'plum', 'pear', 'kiwi'}
run 3  {'fig', 'kiwi', 'pear', 'plum'}
```

one set, three orders

A **set** is a collection of different values, its **elements**, in no order. Write it in braces, or use `set(...)` on any iterable, such as a list or a string. Repeats vanish, so `sorted(set(xs))` lists the distinct items in order. `{}` is an empty **dictionary** (the dictionaries sheet); an empty set is `set()`. Never rely on the printed order.

7 `in` on a set looks in one place



On a list, `x in xs` checks item after item, so the time grows with the list. A set turns a value into a number, its **hash**, which picks the one slot to look in: at a million items it was about 460,000 times faster here (the notebook tried 10 million). A value whose hash can never change is **hashable**: numbers, strings, tuples, not lists.

8 `add`, `remove` and `discard` change a set

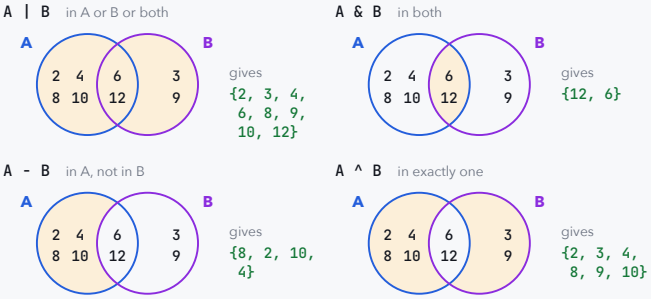
line	s afterwards	what happened
`s = {2, 3, 5}`	`{2, 3, 5}`	make a set
`s.add(7)`	`{2, 3, 5, 7}`	7 added
`s.add(3)`	`{2, 3, 5, 7}`	3 was there: no change
`s.discard(9)`	`{2, 3, 5, 7}`	9 not there: no error
`s.remove(2)`	`{3, 5, 7}`	2 taken out
`s.remove(9)`	`{3, 5, 7}`	9 not there: KeyError: 9

A set comprehension: braces instead of brackets

```
{len(w) for w in ["fig", "kiwi", "pear", "plum"]}  → {3, 4}
[len(w) for w in ["fig", "kiwi", "pear", "plum"]]  → [3, 4, 4, 4]
```

A set can change. `add` puts a value in; adding one already there changes nothing. `remove` takes a value out and raises a **KeyError** if it is missing; `discard` then does nothing. A **set comprehension** is a list comprehension in braces and keeps each result once. `len(s)` counts; `for x in s` visits each element once.

9 Union, intersection and differences



A holds the even numbers up to 12, B the multiples of 3. `|` keeps what is in either, `&` what is in both, `-` what is in the first but not the second, `^` what is in exactly one. Each has a method too, as `A.union(B)`. Order matters for `-`: `B - A` is `{9, 3}`.

10 Common mistakes, with `s = {3, 1, 2}`

you wrote	the last line Python prints, or what you see	why, and the fix
`s = {}; s.add(1)`	`AttributeError: 'dict' object has no attribute 'add'`	`{}` is a dict; `s = set()`
`s[0]`	`TypeError: 'set' object is not subscriptable`	no positions; `sorted(s)[0]` gives 1
`s.remove(9)`	`KeyError: 9`	`s.discard(9)`, or test `9 in s` first
`s.append(4)`	`AttributeError: 'set' object has no attribute 'append'`	sets add: `{1, 2, 3, 4}`
`{[1, 2], [3, 4]}`	`TypeError: unhashable type: 'list'`	use tuples: `{(1, 2), (3, 4)}`
`set("red tan")`	prints `{'d', ' ', 'n', 'r', 't', 'a', 'e'}`	split first: `{'red', 'tan'}`

11 Try it (answers in green)

1. `len(set("banana"))`?
   3: b, a and n.
2. `{n % 4 for n in range(10)}`?
   `{0, 1, 2, 3}`
3. `{1, 2, 3} ^ {2, 3, 4}`?
   `{1, 4}`: in exactly one.
4. `s = {1, 2}`, `s.add(2)`, `s.discard(5)`: `len(s)`?
   2: 2 was there, 5 was not.
5. `{} == set()`? `type({})`?
   `False` and `dict`
6. `max = 3`, then `max(1, 2)`?
   `TypeError: 'int' object is not callable`

names say what · one job per function · comments say why · a set: each value once, no order, fast `in`

Source: the readability notebook and its handwritten notes (essential and extra features), nipunbatra.github.io/comp22; the other examples are our own. Every output and error comes from running the code with Python 3.11.11 (style report: ruff 0.12.1; timings from one machine, they vary). Next: the dictionaries sheet.